



Universidad de Jaén

Bloque 3

Ingeniería de Requisitos



EPS
Escuela Politécnica
Superior de Jaén

Objetivos
Introducción
Diagramas de secuencia
Diagramas de comunicación
Diagramas generales de interacción
Diagramas de tiempos
Consistencia del modelo

Tema 6. Modelado de la Dinámica

L. Alfonso Ureña López Eugenio Martínez Cámara

Departamento de Informática
Universidad de Jaén

Fundamentos de Ingeniería del Software
Grado en Ingeniería Informática
2º Curso

Contenidos

- 1 Objetivos
- 2 Introducción
 - Interacción y Colaboración
 - Modelado de las interacciones
- 3 Diagramas de secuencia
 - Propiedades
 - Representación
 - Guía para elaborar diagramas de secuencia
- 4 Diagramas de comunicación
 - Propiedades
 - Ejemplos
 - DS vs DC
- 5 Diagramas generales de interacción
 - Propiedades
 - Ejemplo
- 6 Diagramas de tiempos
 - Propiedades
 - Ejemplos

Objetivos

- Conocer y diferenciar los conceptos de colaboración e interacción de objetos
- Aprender a modelar la interacción de objetos mediante
 - Diagramas de secuencia de interacción
 - Diagramas de comunicación
 - Diagramas generales de interacción
 - Diagramas de tiempo
- Entender las diferencias y semejanzas entre los distintos tipos de diagramas de modelado analizados
- Adquirir la capacidad de decidir qué tipos de diagramas son más adecuados en cada situación

Interacción y Colaboración



Colaboración

Una **colaboración** es un grupo de objetos o clases que trabajan juntas para proporcionar un elemento de funcionalidad o comportamiento



Interacción

Una **interacción** define el paso de mensajes entre líneas de vida (por ejemplo, objetos) en el contexto de una colaboración para lograr un comportamiento particular

Modelado de las interacciones

Las interacciones se pueden modelar usando varias notaciones:



- Diagramas de secuencia
- Diagramas de comunicación
- Diagramas de tiempo
- Diagramas generales de la interacción

Propiedades

- Muestran una **interacción entre objetos** organizados en una **secuencia de tiempo**
- Los diagramas de secuencia pueden dibujarse con **distintos niveles de detalle** y para satisfacer distintos objetivos en las **diversas etapas del ciclo de vida** del desarrollo del sistema
- Normalmente se utilizan para **representar** la interacción entre objetos que se produce en un **caso de uso** o para una operación
- Cuando se utiliza para modelar el comportamiento dinámico de un caso de uso puede considerarse como una **especificación detallada del caso de uso**

Representación general

- La **dimensión vertical** representa el **tiempo**
- Los **objetos involucrados** en la interacción están distribuidos **horizontalmente** en el diagrama
- La **ordenación horizontal** de los objetos es arbitraria, aunque mejora la legibilidad si la **interacción progresa de izquierda a derecha**
- Cada **objeto** está representado por una **línea de vida**, que se representa por una **línea vertical discontinua** y con un símbolo de objeto en su parte superior
- Un **mensaje** se representa por una **flecha horizontal** que va **desde una línea de vida a otra**

Representación general

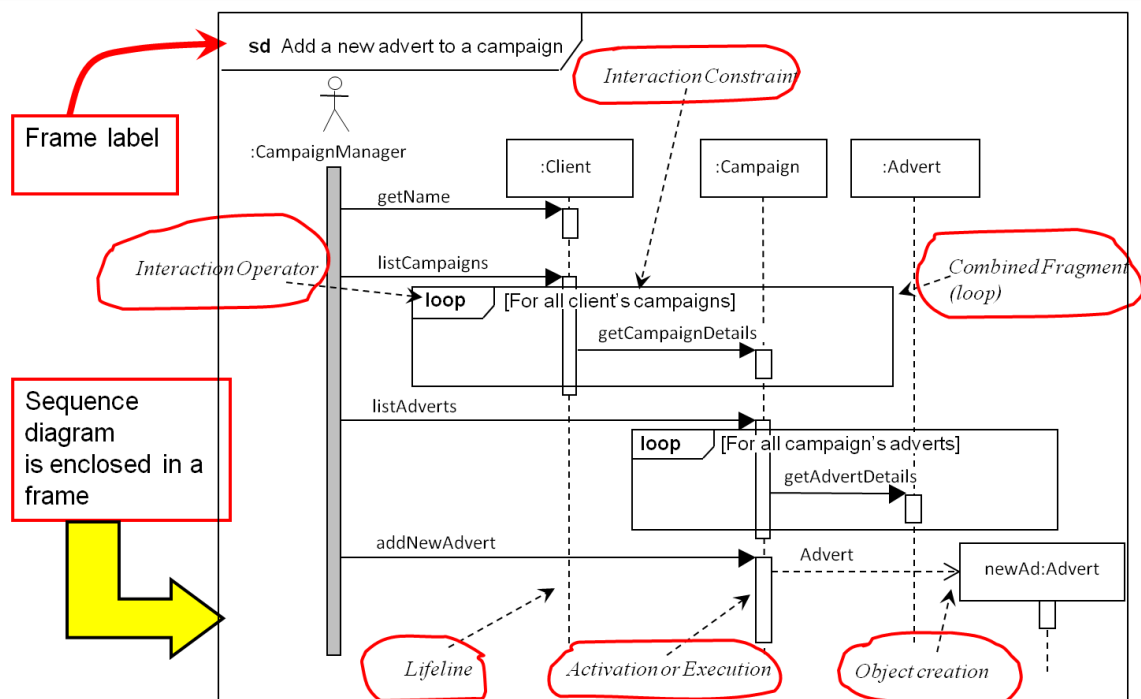
- Cuando se **envía un mensaje** a un objeto se está **llamando** a una **operación de ese objeto**
- El **nombre del mensaje** generalmente **coincide** con el de la **operación** que se está llamando
- El **periodo de tiempo** durante el que se está ejecutando una **operación** se conoce como una **ocurrencia de ejecución o de activación**
- Dicho **periodo de tiempo** se representa mediante un **bloque rectangular** situado a lo largo de la **línea de vida**

Representación general

- Los diagramas de secuencia se representan dentro de un rectángulo denominado **marco**
- El **nombre del marco** puede corresponderse con el **nombre del caso de uso** representado
- La **iteración** se **representa** dentro de un **marco** titulado con el nombre de la interacción
- La interacción dentro de este tipo de marco se conoce como **fragmento combinado**, donde la palabra clave **loop** es un ejemplo de un **operador de interacción**

Representación general

- Un **fragmento combinado** con una limitación de interacción sólo se ejecutará si la limitación se evalúa como verdadera
- La **creación de un objeto** es mostrada mediante una **flecha de construcción** (discontinua) **apuntando** directamente al símbolo del **objeto creado**
- Cuando el **objeto ya existe**, la **flecha** del mensaje **apuntará** directamente sobre algún punto de la **línea de vida** del objeto de destino



Clases límite y de control



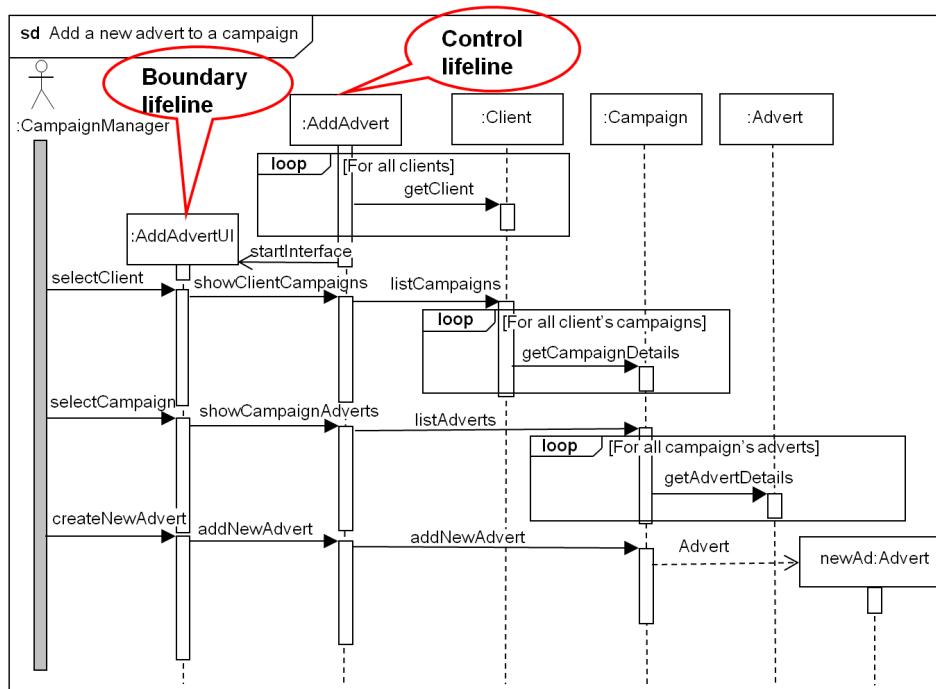
Clases límite

- La **mayoría de los casos de uso** implican al menos un **objeto límite** que establezca el diálogo **entre el actor y el sistema**
- En el diagrama siguiente esto es representado por la **línea de vida :AddAdvertUI**



Clases de control

- El objeto control es representado por la **línea de vida :AddAdvert**
- Se encarga de **gestionar la comunicación** con los demás objetos

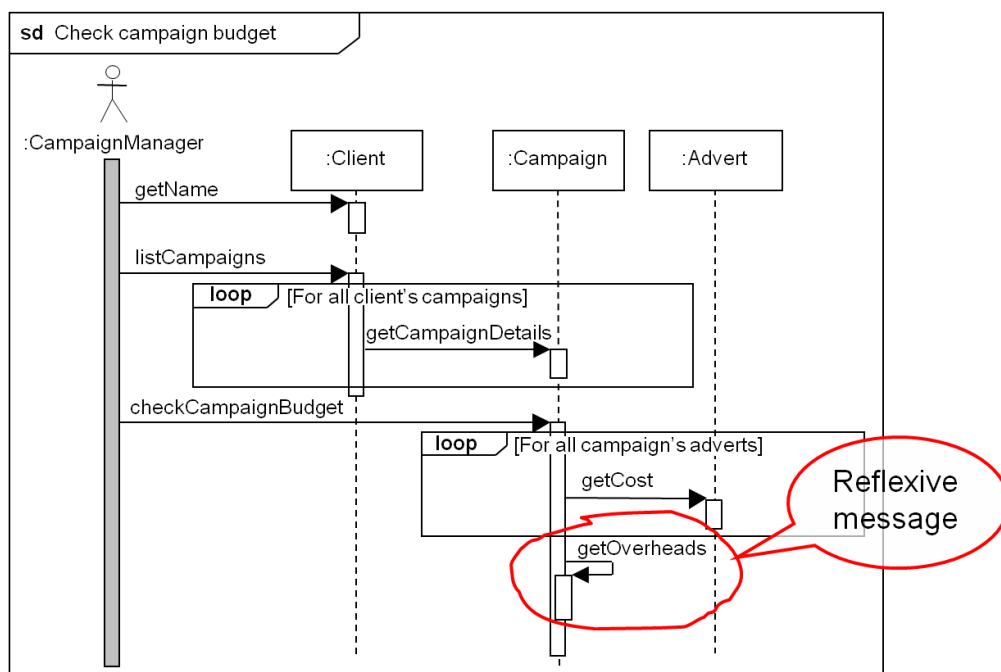


Uso de mensajes reflexivos



Mensajes reflexivos

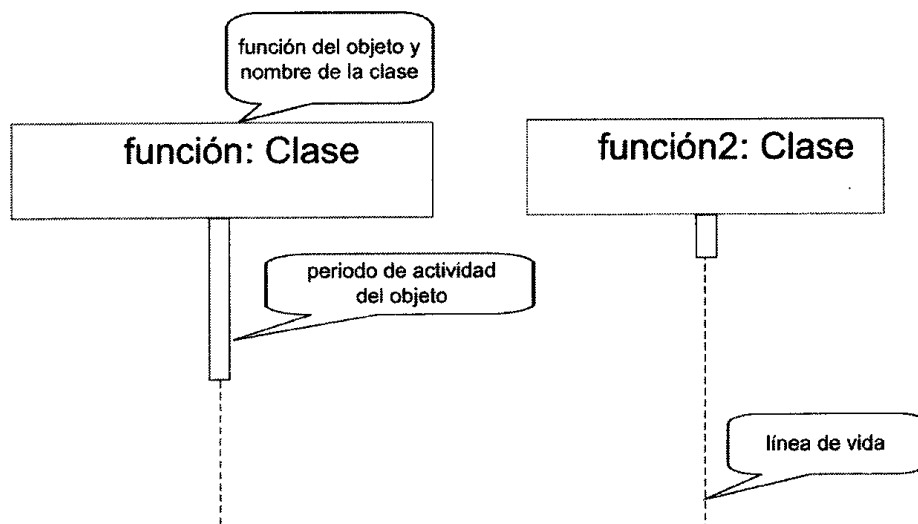
Se trata de mensajes que se envía un objeto a sí mismo



Línea de vida de un objeto

- El diagrama de secuencia hace entrar en acción las instancias de las clases involucradas en la realización de la subfunción a la que está vinculado
- A **cada instancia** se le asocia **una línea de vida**:
 - Muestra las **acciones y reacciones**
 - Muestra los **periodos** durante los cuales está **activa**
 - En cada **periodo de actividad** ejecuta alguno de sus **métodos**

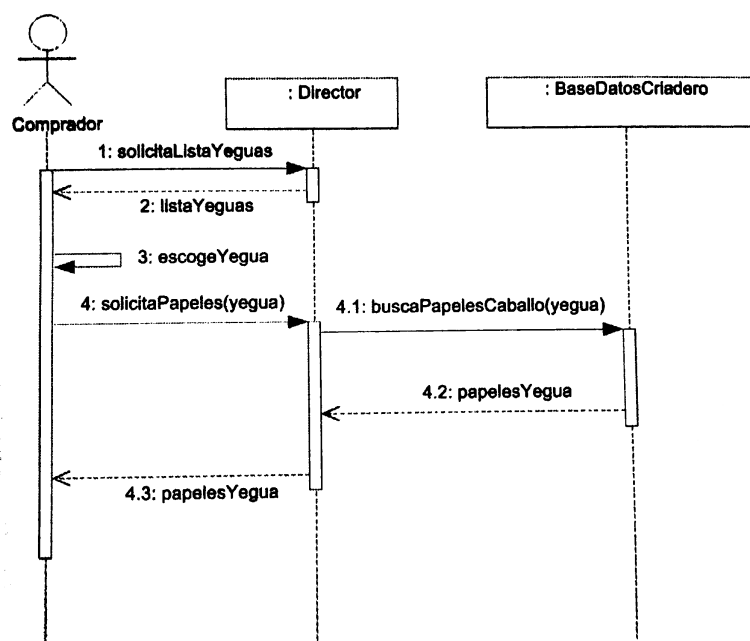
Línea de vida de un objeto



Envío de mensajes

- Se representan mediante **flechas horizontales** que unen la línea de vida del **objeto emisor** con la línea de vida del **objeto destinatario**
- Los mensajes se pueden **numerar secuencialmente** a partir del valor **uno**
- Si un mensaje se envía antes de que concluya el tratamiento del precedente, **es posible utilizar una numeración compuesta**
- La **numeración no es obligatoria**, pero resulta **práctica** especialmente para mostrar las **activaciones anidadas**

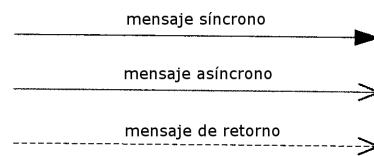
Envío de mensajes



Tipos de mensajes



- Síncronos
- Asíncronos
- De retorno



Tipos de mensajes



Mensaje síncrono

- También se denomina **llamada a procedimiento**
- Es el tipo de **mensaje que más se utiliza**
- Su uso implica que el **expedidor del mensaje espera** que la activación del método asociado al **destinatario finalice** para poder continuar su actividad

Tipos de mensajes



Mensaje de retorno

- Se encargan de **devolver el control** al **objeto** que originó el mensaje que inició la activación
- Se representan mediante una **flecha con línea discontinua**
- Su uso es **opcional**, ya que podemos asumir que el control siempre es devuelto al objeto que originó el mensaje a la finalización de la ocurrencia de ejecución

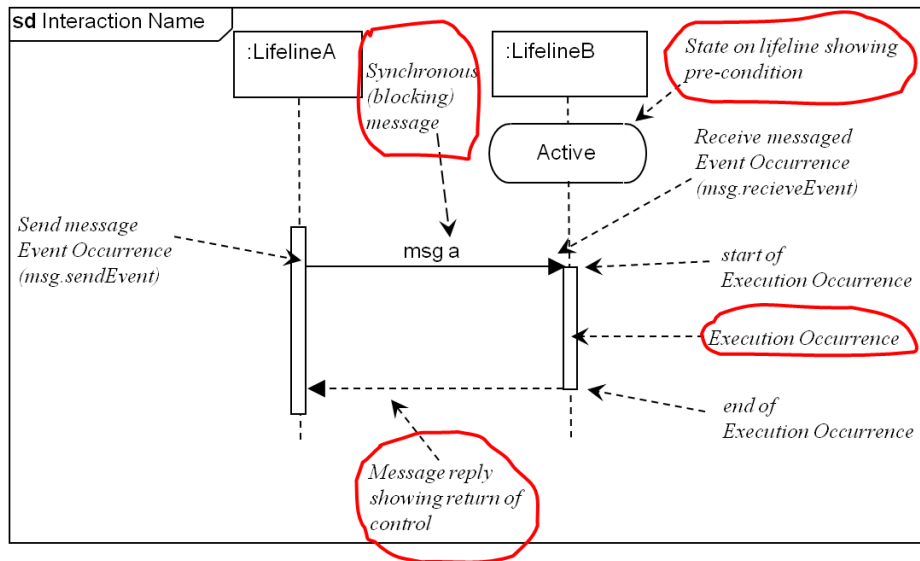
Tipos de mensajes



Mensaje asíncrono

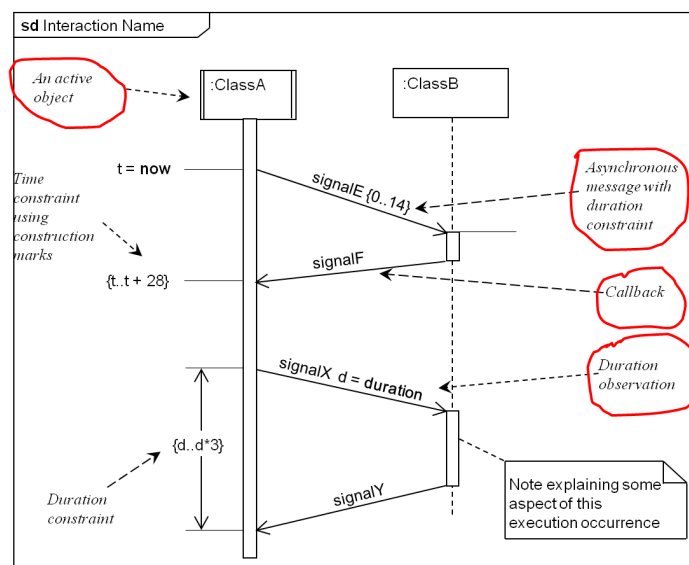
- El **expedidor no espera** el término de la activación invocada por el **destinatario**
- Esto se produce al modelar **sistemas** en los que los **objetos** pueden funcionar **en paralelo**

Utilizando mensajes síncronos y de retorno...



© 2010 Bennett, McRobb and Farmer

Utilizando mensajes asíncronos...

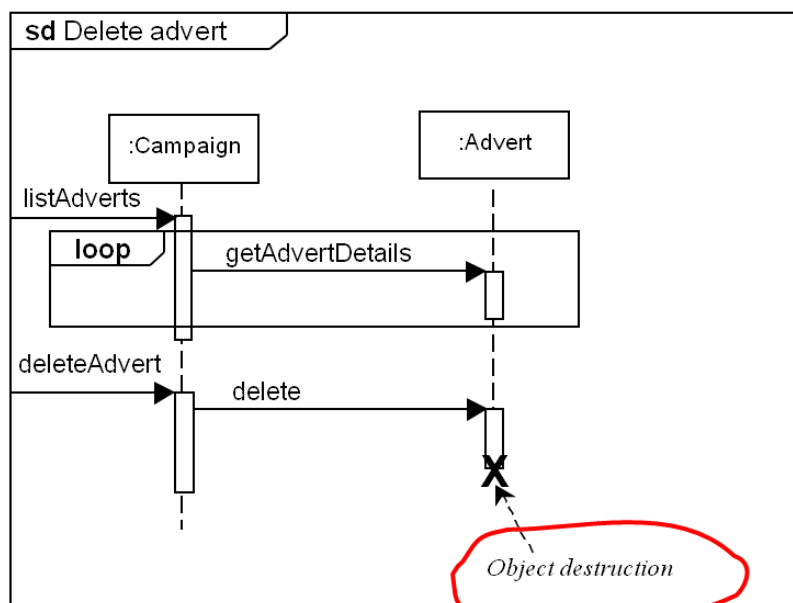


© 2010 Bennett, McRobb and Farmer

Destrucción de objetos

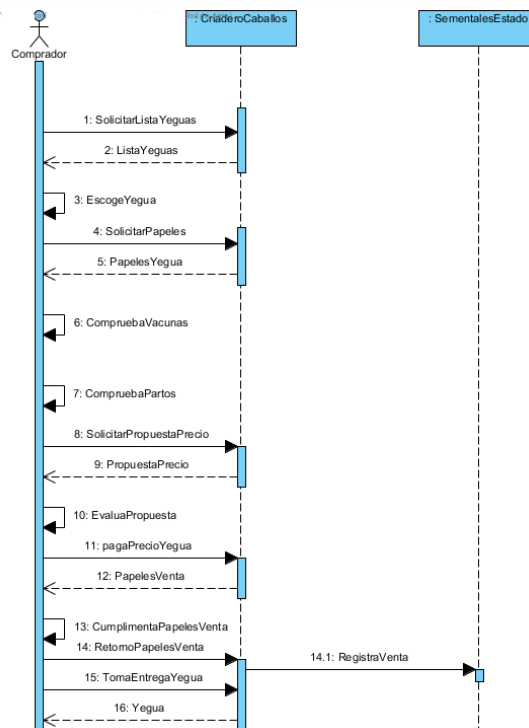
- Se trata de un **mensaje** enviado a un objeto existente y que da lugar a la **finalización de su línea de vida**
- Se representa mediante una **cruz al final de su línea de vida**

Destrucción de objetos



Ejemplo de representación

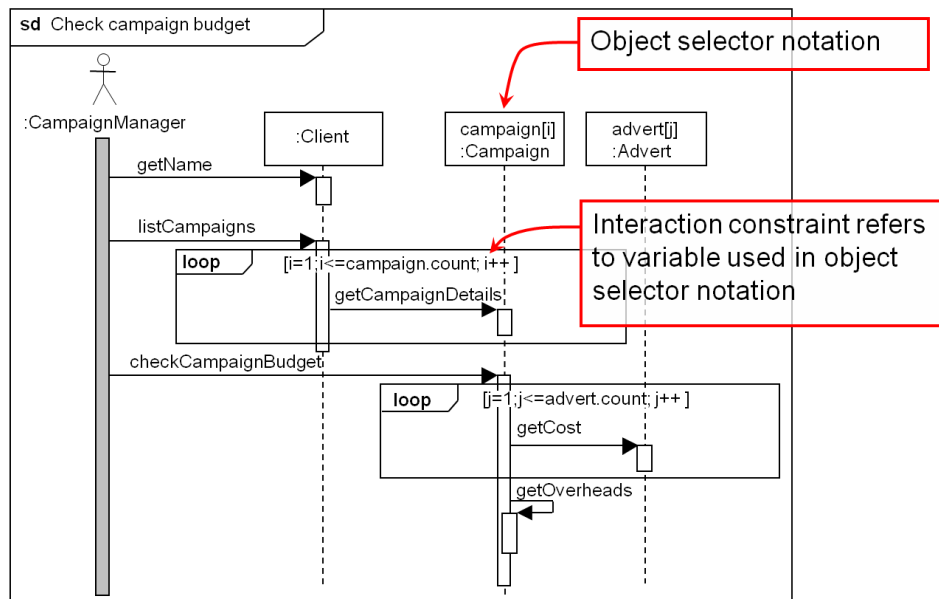
Compra de una yegua



Fragmentos combinados: Operadores de Interacción

Interaction Operator	Explanation and use
alt	Alternatives represents alternative behaviours, each choice of behaviour being shown in a separate operand. The operand whose interaction constraint is evaluated as true executes.
opt	Option describes a single choice of operand that will only execute if its interaction constraint evaluates as true.
break	Break indicates that the combined fragment is performed instead of the remainder of the enclosing interaction fragment.
par	Parallel indicates that the execution operands in the combined fragment may be merged in any sequence once the event sequence in each operand is preserved.
seq	Weak Sequencing results in the ordering of each operand being maintained but event occurrence from different operands on different lifelines may occur in any order. The order of event occurrences on common operands is the same as the order of the operands.
strict	Strict Sequencing imposes a strict sequence on execution of the operands but does not apply to nested fragments.
neg	Negative describes an operand that is invalid.
critical	Critical Region imposes a constraint on the operand that none of its event occurrences on the lifelines in the region can be interleaved.
ignore	Ignore indicates the message types, specified as parameters, that should be ignored in the interaction.
consider	Consider states which messages should be consider in the interaction. This is equivalent to stating that all others should be ignored.
assert	Assertion states that the sequence of messaging in the operand is the only valid continuation.
loop	Loop is used to indicate an operand that is repeated a number times until the interaction constraint for the loop is no longer true.

Fragmentos combinados: Referenciando objetos desde la condición de control



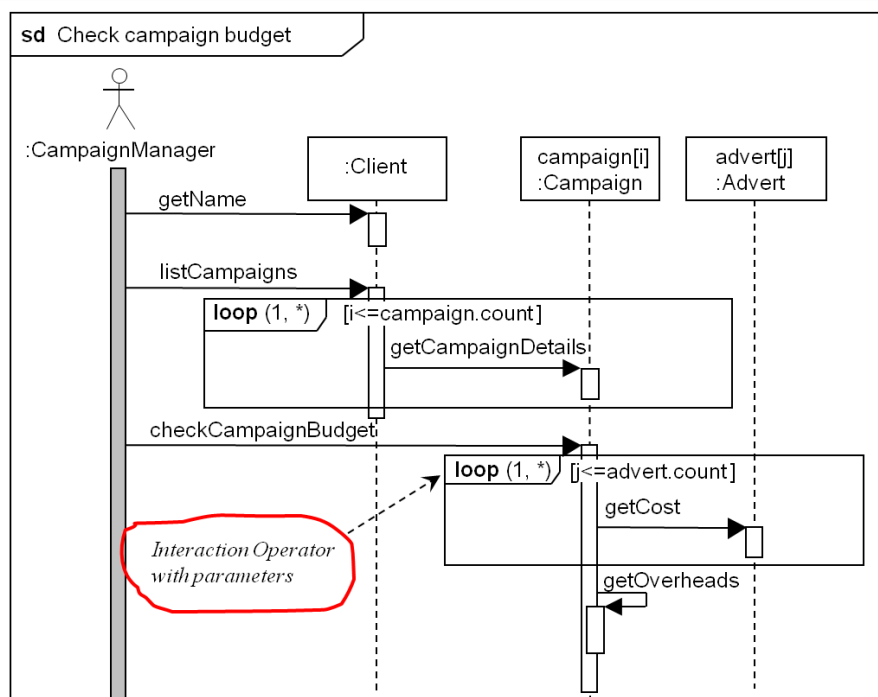
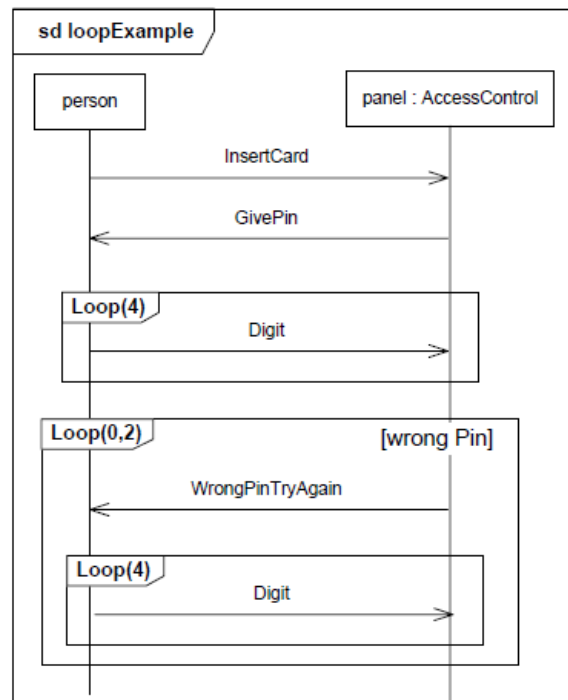
© 2010 Bennett, McRobb and Farmer

Fragmentos combinados: Uso de parámetros en los operadores de iteración



Operador loop

- Es el único operador que admite parámetros
- Sintaxis:
 - **loop**['(' <minint> [, <maxint>] ')']
 - '<minint>' ',' '<maxint>' ::= enteros no negativos
- Ejemplos:
 - **loop(4)** ⇨ ejecuta el fragmento combinado 4 veces
 - **loop(1, 3)** ⇨ ejecuta el fragmento combinado 3 veces
 - **loop(1, *)** ⇨ ejecuta el fragmento indefinidamente
 - Si el fragmento combinado del operador loop tiene una condición de control entonces ésta debe ser cierta en cada iteración para que se ejecute

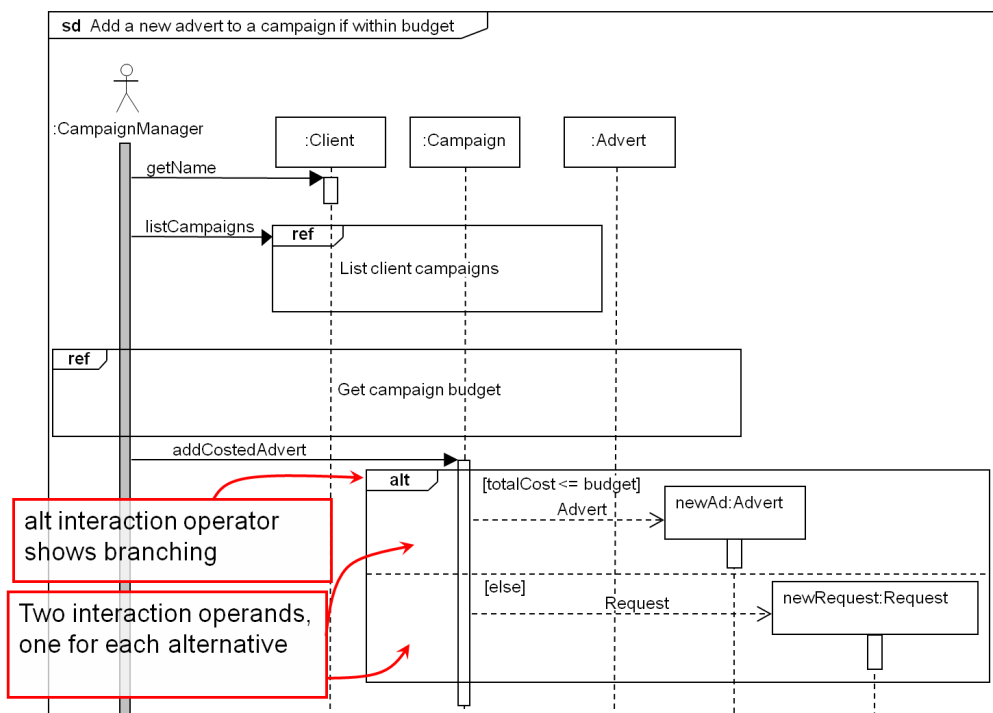


Fragmentos combinados: Uso de alternativas



Operador alt

- Es el único operador de interacción que permite ejecutar distintas alternativas en un fragmento combinado
- Siempre debe poder **ejecutarse una y sólo una de las alternativas**, lo cual se debe controlar con las condiciones de control apropiadas en cada una de ellas
- Opcionalmente, podrá incluir una condición **else**

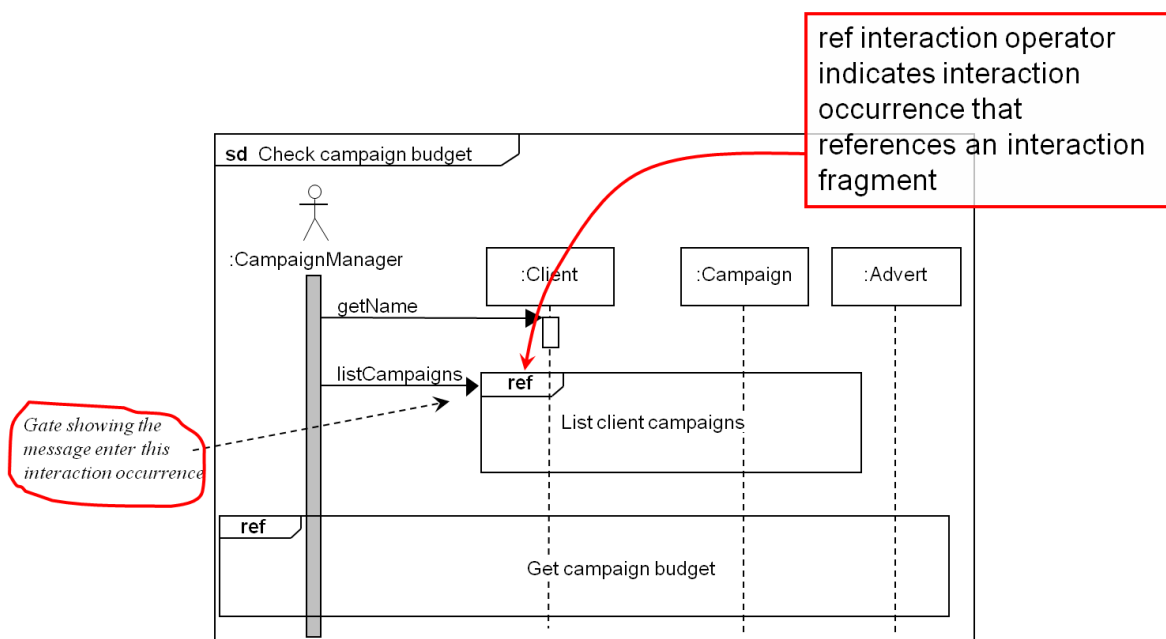


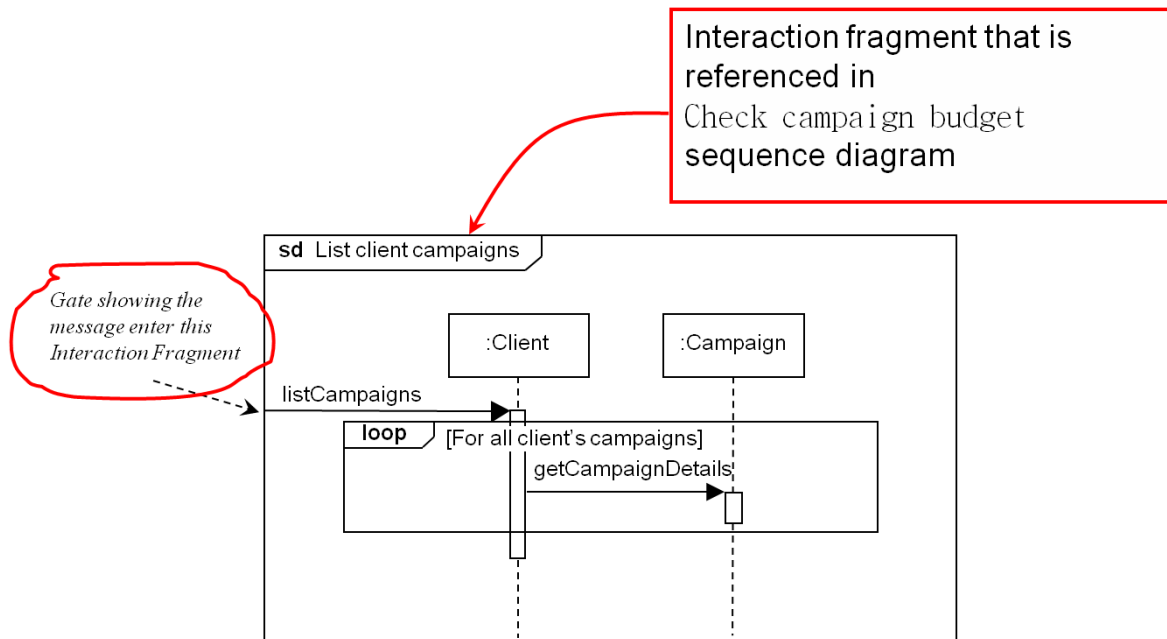
Fragmentos combinados: Representación de interacciones complejas



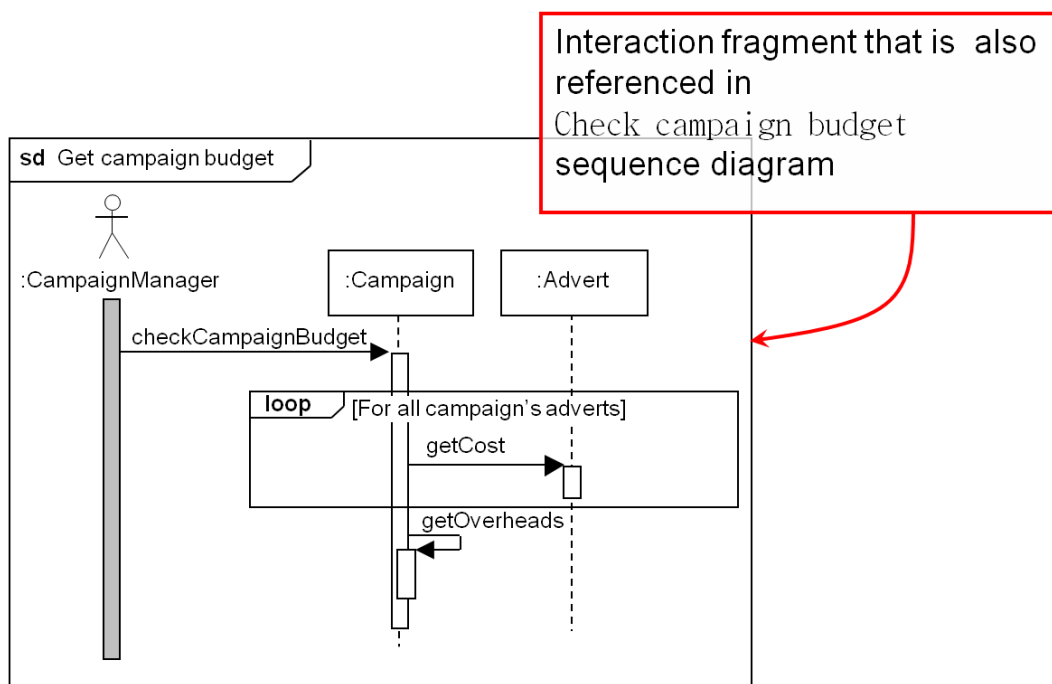
Operador ref

- Se utiliza para representar interacciones complejas
- El uso del operador **ref** permite simplificar la representación de un diagrama de secuencia de interacción, referenciando desde el correspondiente fragmento combinado otros fragmentos de interacción





© 2010 Bennett, McRobb and Farmer



© 2010 Bennett, McRobb and Farmer

Guía para elaborar diagramas de secuencia

- 1 Decidir a qué **nivel** se está modelando la **interacción**
- 2 **Identificar** los principales **elementos** implicados en la **interacción**
- 3 **Considerar** los **escenarios alternativos** que puedan ser necesarios
- 4 **Identificar** cualquier **interacción existente** que haya sido modelada como diagramas de secuencia, **o que lo vaya a ser** para que puedan incluirse como ocurrencias de interacción
- 5 **Dibujar** el esquema de la **estructura** del diagrama
- 6 Añadir la **interacción detallada**
- 7 **Comprobar la consistencia** con los **diagramas de secuencia vinculados** y modificarlos según sea necesario
- 8 **Comprobar la consistencia** con **otros diagramas** o modelos UML

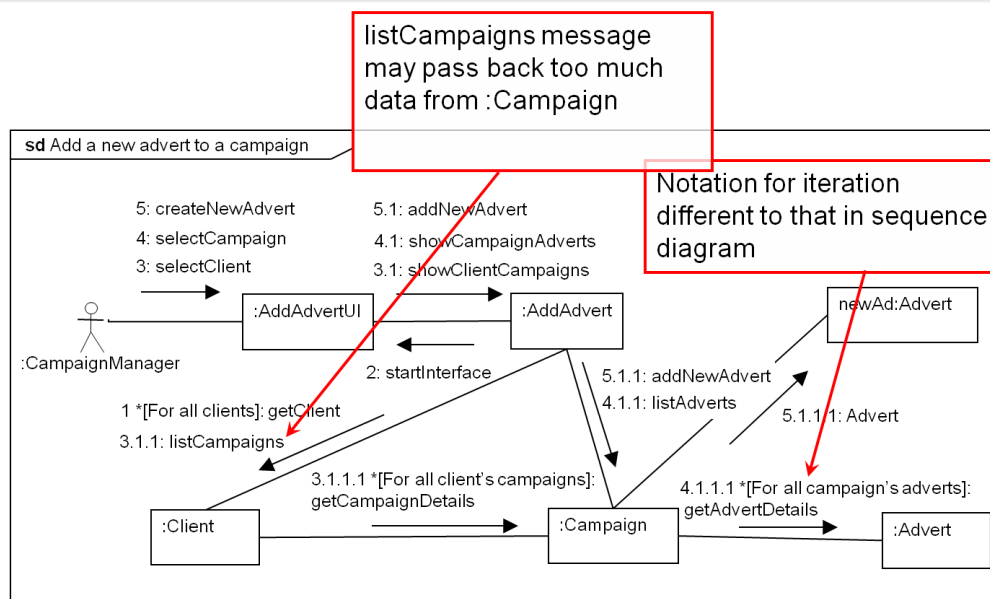
Propiedades

- Los diagramas de comunicación (DC) mantienen la misma información que los diagramas de secuencia (DS)
- Muestran los vínculos entre los objetos que participan en la colaboración
- Ninguna dimensión del tiempo, la secuencia se captura con números de secuencia
- Los números de secuencia se escriben en un estilo anidado (por ejemplo, 3.1 y 3.1.1) para indicar la anidación de control dentro de la interacción que se está modelando

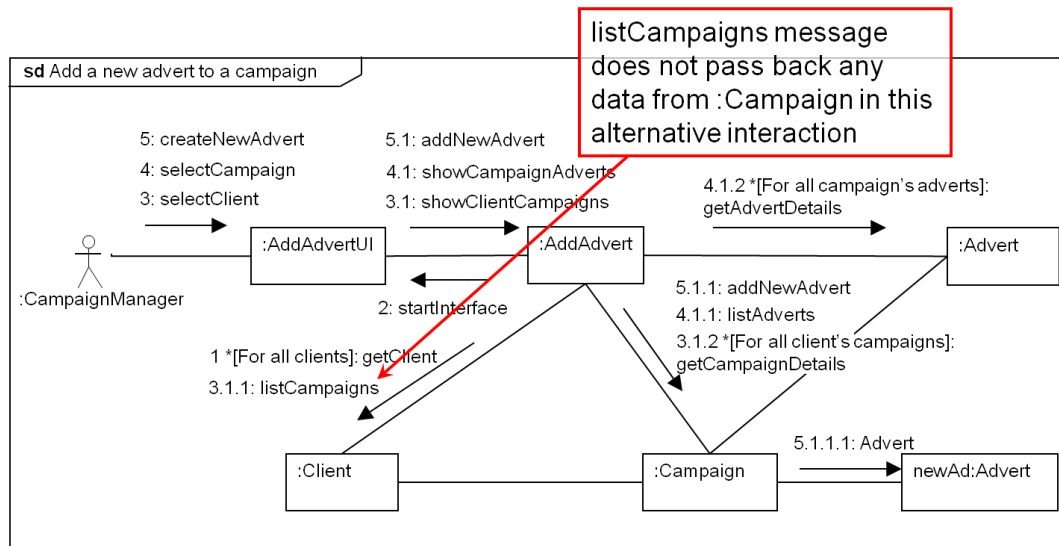
Etiquetas de mensaje

Tipo de mensaje	Ejemplo de sintaxis
Mensaje sencillo	4: addNewAdvert
Llamada anidada con valor de retorno. <i>El valor devuelto se guarda en la variable name.</i>	3.1.2: name = getName
Mensaje condicional. <i>Este mensaje se envía solamente si la condición [balance > 0] es verdadera</i>	5 [balance > 0]: debit(amount)
Iteración con ejecución secuencial de los mensajes	4.1 *[For all adverts]: getCost
Iteración con ejecución paralela de los mensajes	4.1 * [For all adverts]: getCost

Ejemplo de diagrama de comunicación

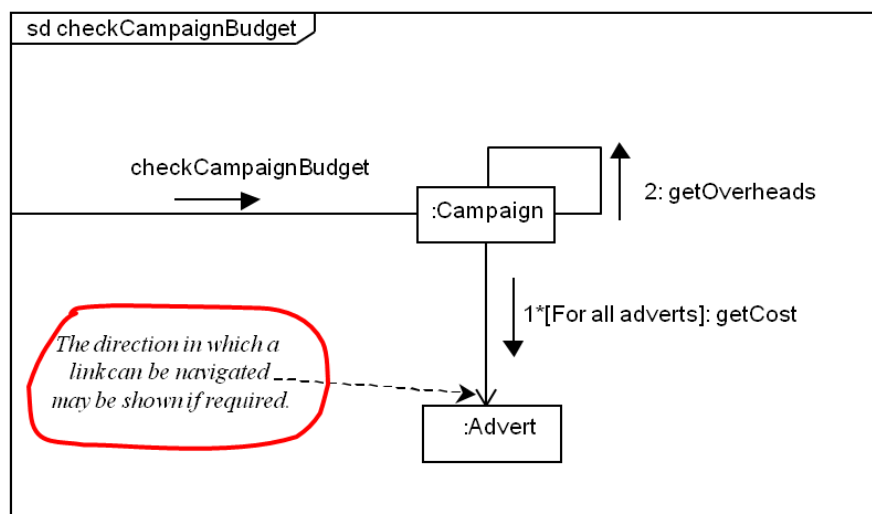


Ejemplo de diagrama de comunicación



© 2010 Bennett, McRobb and Farmer

Ejemplo de dirección explícita de navegación



© 2010 Bennett, McRobb and Farmer

DS vs DC

- Los DS y los DC tienen muchas similitudes
- Principales diferencias:
 - Los DC no proporcionan el mismo nivel de sintaxis que los DS y no son adecuados para interacciones complejas
 - Los DC se suelen utilizar para describir casos de uso de análisis, por no estar aún completamente especificados los mensajes
 - Cuando hay muchos mensajes entre dos objetos un DC es más difícil de leer que el DS equivalente
 - Los DS ofrecen más indicaciones visuales explícitas de la duración de cada activación
 - Los DS muestran con mayor claridad las interacciones de diseño detalladas

Conclusiones

- Algunos diseñadores prefieren los DC a los DS porque ofrecen una visión de interacción del objeto que es más fácil de relacionar con la colaboración subyacente a causa de la visibilidad de los enlaces entre líneas de vida
- Raramente es útil dibujar ambos tipos de diagrama de interacción para la misma finalidad
- Algunos diseñadores utilizan los:
 - DS para el modelo genérico de la interacción
 - DC para los escenarios específicos
- La elección del más apropiado depende de la naturaleza de la interacción y la finalidad del diagrama

Propiedades

- Los diagramas generales de interacción (DGI) han sido presentados en **UML 2**
- Se centran en la **visión de conjunto del flujo de control** en una interacción
- Los **nodos** del diagrama pueden ser **interacciones u ocurrencias de interacción**
- Para generar un diagrama general de interacción es necesario **desglosar** la **interacción** en sus **elementos clave**

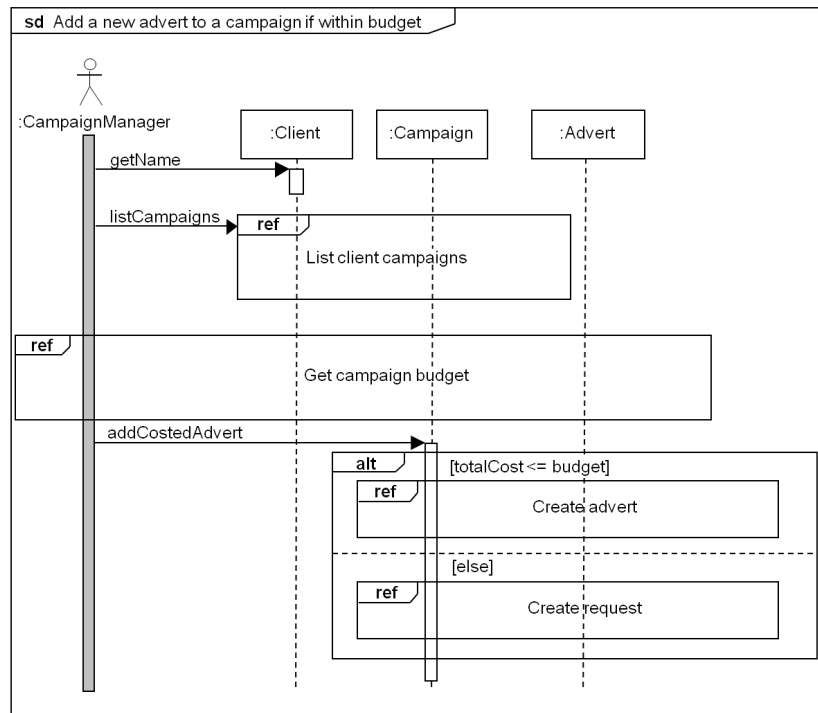
Ejemplo



A continuación se propone una versión alternativa del diagrama de secuencia

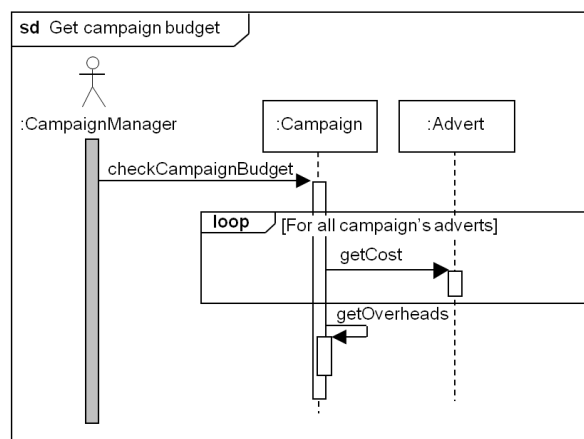
Add a new advert to a campaign if within budget

para ser usado en el desarrollo del diagrama general de interacción



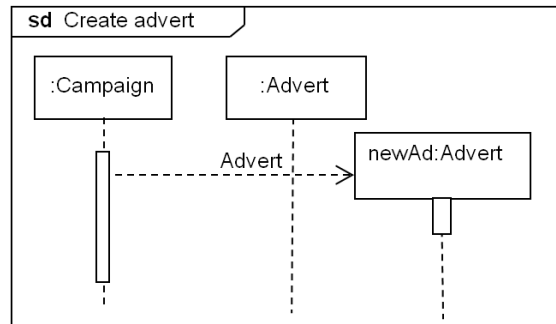
© Bennett, McRobb and Farmer 2010

Fragmento de interacción Get campaign budget



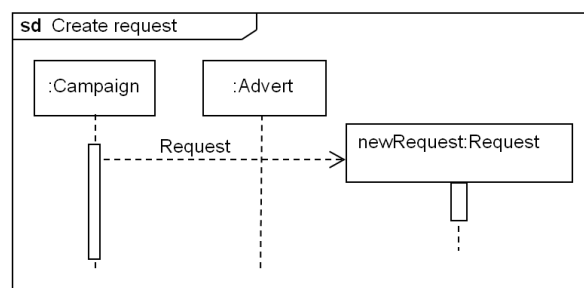
© Bennett, McRobb and Farmer 2010

Fragmento de interacción Create advert

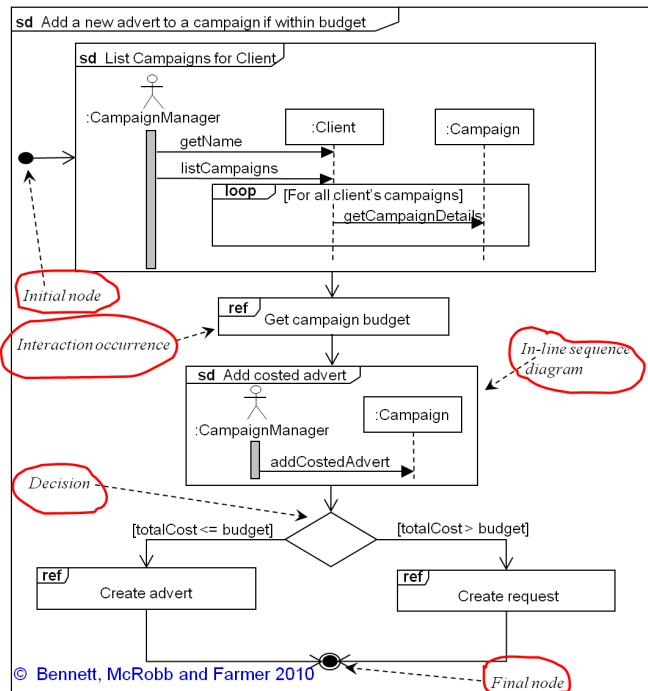


© Bennett, McRobb and Farmer 2010

Fragmento de interacción Create request



© Bennett, McRobb and Farmer 2010



Propiedades

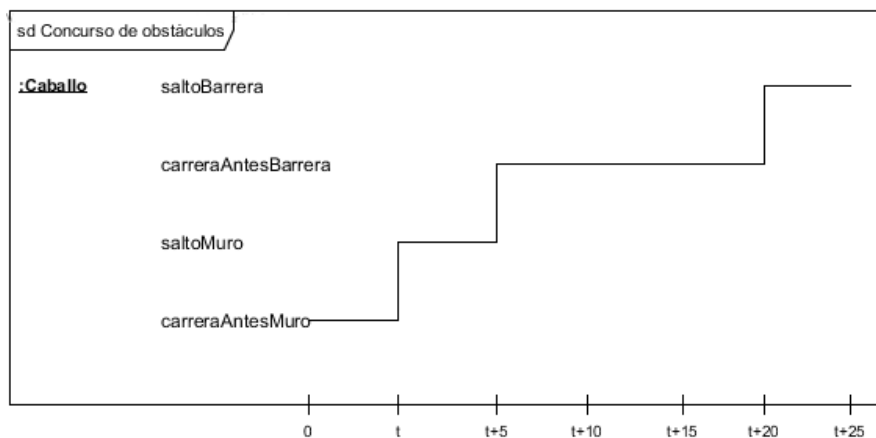
- Nueva característica en **UML 2**
- Muestran los **cambios de estado** de un objeto cuando éstos **dependen exclusivamente del tiempo**
- Muestran cómo las **limitaciones de tiempo afectan** a las **interacciones entre líneas de vida**



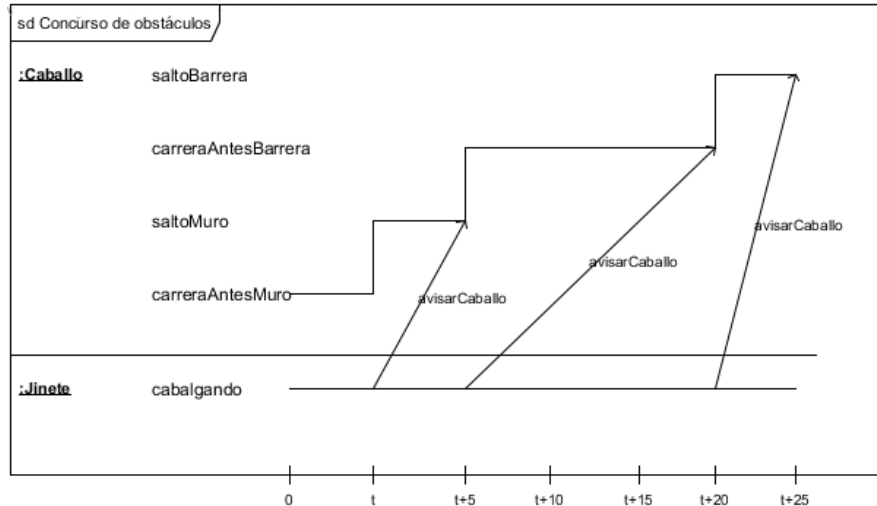
Concurso de obstáculos

En un concurso de obstáculos, el jinete no debe superar un tiempo máximo para realizar la totalidad de la prueba, de lo contrario quedará eliminado. Él mismo descompone el tiempo de cada prueba para estar seguro de superarla con éxito

Concurso de obstáculos: sin paso de mensajes



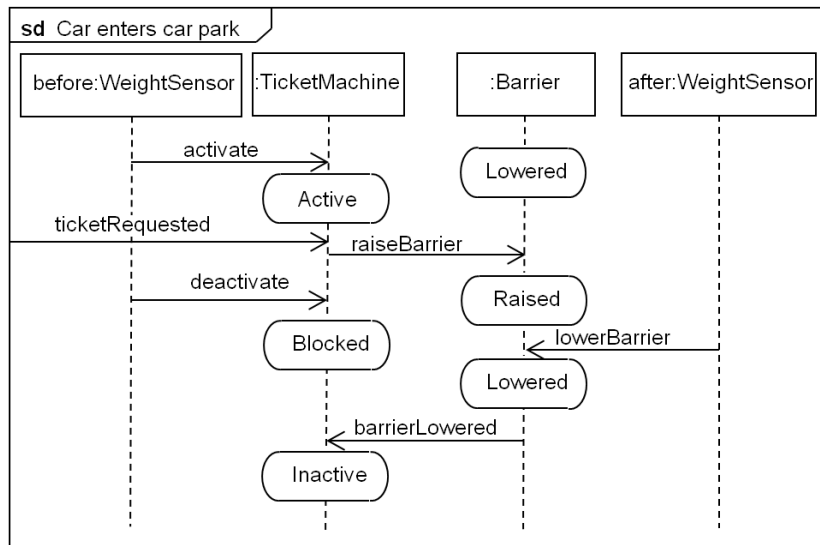
Concurso de obstáculos: con paso de mensajes



Control de acceso a un parking

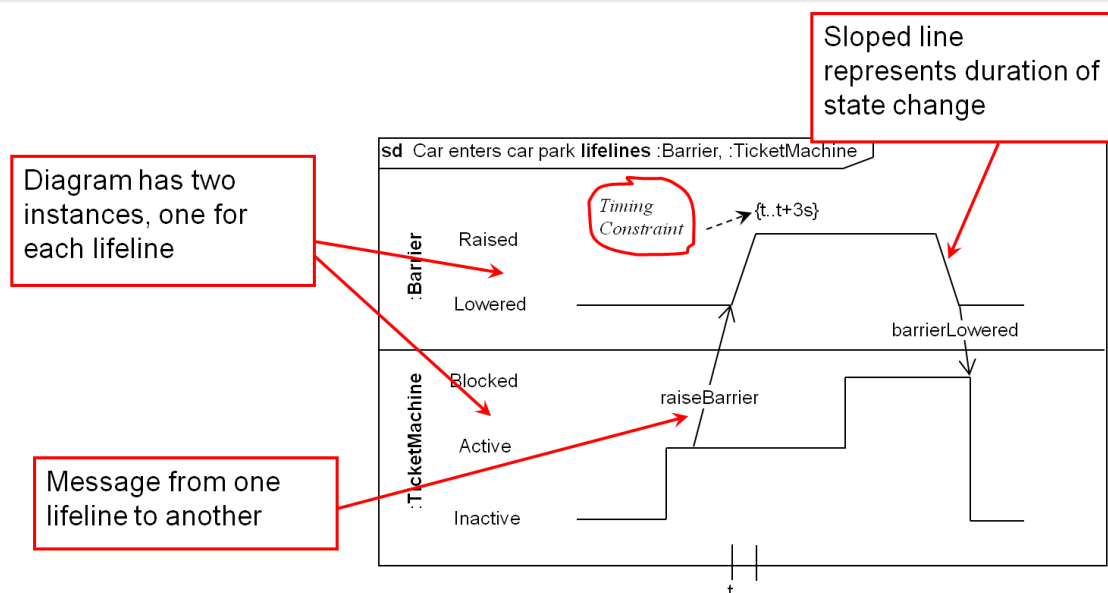
- En este ejemplo se construye un diagrama de tiempos a partir de un diagrama de secuencia de interacción que muestra el control de acceso a un parking de vehículos
- Los diagramas de tiempo deben ser consistentes con los diagramas de secuencia relevantes

Control de acceso a parking: Diagrama de secuencia



© Bennett, McRobb and Farmer 2010

Control de acceso a parking: Diagrama de tiempos



© Bennett, McRobb and Farmer 2010

Consistencia del modelo

- La preparación de los diagramas de secuencia y de comunicación implica la adjudicación de operaciones a clases
- Los diagramas de secuencia y de comunicación y los diagramas de clases deberían ser mutuamente consistentes
- Una buena herramienta CASE refuerza esta consistencia a nivel sintáctico, mostrando al diseñador una lista de operaciones adjudicadas a una clase cuando trata de enviar un mensaje a un objeto de esa clase
- Pero para asegurar una consistencia completa entre un diagrama de clase y un conjunto de diagramas de interacción se necesita más que una simple comprobación sintáctica

Consistencia del modelo

- Cada objeto remitente debe tener la capacidad de enviar su mensaje al objeto destino y para ello necesita conocer la identidad o *referencia de objeto* del objeto destino
- Sólo hay dos formas de que un objeto remitente pueda conocer la referencia del objeto destino:
 - Mediante un enlace directo al objeto destino $\rightsquigarrow$ existe una asociación entre las clases implicadas
 - De forma indirecta a través de otro objeto (generalmente de otra clase) que tiene a su vez relación con el objeto destino $\rightsquigarrow$ existe una ruta de asociaciones entre las clases implicadas
- Cualquier inconsistencia entre un diagrama de interacción y el diagrama de clase correspondiente indica un fallo en alguno de ellos

Bibliografía



S. Bennet et al.

Análisis y Diseño Orientado a Objetos de Sistemas Usando UML. 3ª Ed.

McGraw-Hill, 2007



L. Debrauwer and F. Van der Heyde

UML 2. Iniciación, Ejemplos y Ejercicios Corregidos. 3ª Ed.

Ediciones ENI, 2013

Enlaces de interés



<http://www.uml-diagrams.org/sequence-diagrams.html>

UML Sequence Diagrams



<http://www.uml-diagrams.org/communication-diagrams.html>

UML Communication Diagrams Overview



<http://www.uml-diagrams.org/interaction-overview-diagrams.html>

UML Interaction Overview Diagrams



<http://www.uml-diagrams.org/timing-diagrams.html>

Timing Diagrams